

Lab 3 - Terraform + Ansible + Docker: Single Containerized App

Full Lab Guide, Configuration & Source Code

Terraform / Ansible / Docker / Kubernetes Lab Series

Contents

Use Case	1
Architecture	1
Directory Layout	2
Prerequisites	2
Step-by-Step Configuration & Execution	2
Step 1 — Provision infrastructure with Terraform	2
Step 2 — Install required Ansible collection	2
Step 3 — Wait for SSH to be ready, then run the playbook	3
Step 4 — Confirm the app is live	3
Validation Steps	3
Cleanup	3
Key Concepts Reinforced	4

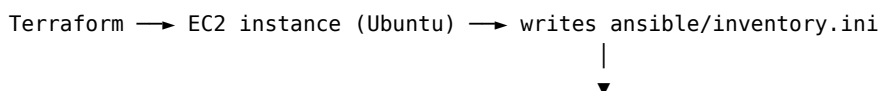
Appendix: Full Source Code

ansible/ansible.cfg	5
ansible/playbook.yml	5
ansible/requirements.yml	5
ansible/roles/deploy_app/tasks/main.yml	5
ansible/roles/deploy_app/templates/Dockerfile.j2	6
ansible/roles/deploy_app/templates/index.html.j2	6
ansible/roles/docker/tasks/main.yml	6
app/Dockerfile	7
app/index.html	7
terraform/main.tf	7
terraform/outputs.tf	9
terraform/variables.tf	9

Use Case

A team wants a repeatable pipeline: **Terraform** provisions the VM, **Ansible** installs and configures Docker (and only Docker — no manual SSH steps), and **Docker** runs the actual application as a container built from a templated Dockerfile. This is the canonical “infra vs. config vs. run-time” separation of concerns pattern.

Architecture



```

Ansible (roles: docker, deploy_app)
    |
    ▼
Docker build + run (nginx:alpine + index.html)
    |
    ▼
curl http://<public_ip> → 200 OK

```

Directory Layout

```

lab3-tf-ansible-docker-singleapp/
├── terraform/
│   ├── main.tf          # EC2 + SG + writes ansible/inventory.ini automatically
│   ├── variables.tf
│   └── outputs.tf
├── ansible/
│   ├── ansible.cfg
│   ├── requirements.yml # community.docker collection
│   ├── playbook.yml
│   └── roles/
│       ├── docker/tasks/main.yml    # installs Docker Engine
│       └── deploy_app/
│           ├── tasks/main.yml        # builds + runs the container
│           └── templates/
│               ├── Dockerfile.j2
│               └── index.html.j2
└── app/
    ├── Dockerfile
    └── index.html
    # standalone reference copy for local `docker build` testing

```

Prerequisites

- Terraform $\geq$ 1.5.0, AWS CLI configured
- Ansible $\geq$ 2.14, ansible-galaxy collection install -r ansible/requirements.yml
- An EC2 key pair + local .pem file

Step-by-Step Configuration & Execution

Step 1 — Provision infrastructure with Terraform

```

cd terraform
terraform init
terraform apply \
  -var="key_name=my-ec2-keypair" \
  -var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem"

```

Terraform will: 1. Create a default-VPC EC2 instance + security group (22, 80 open) 2. **Automatically generate** ../ansible/inventory.ini populated with the instance's public IP, SSH user, and key path — no manual copy/paste

Step 2 — Install required Ansible collection

```

cd ../ansible
ansible-galaxy collection install -r requirements.yml

```

Step 3 — Wait for SSH to be ready, then run the playbook

```
# small wait loop so Ansible doesn't race the boot process
until ansible app_servers -m ping; do sleep 5; done

ansible-playbook playbook.yml
```

The `docker` role installs Docker Engine + Compose plugin; the `deploy_app` role templates a Dockerfile + `index.html` to `/opt/app` on the remote host, then uses `community.docker.docker_image / docker_container` to build and run the container — all idempotently (re-running the playbook is safe and makes no changes if nothing changed).

Step 4 — Confirm the app is live

```
curl http://$(cd ../terraform && terraform output -raw app_server_public_ip)
```

Validation Steps

1. Terraform state matches reality

```
cd terraform && terraform plan # should report "No changes"
```

2. Docker installed correctly

```
ansible app_servers -m shell -a "docker --version && docker compose version"
```

3. Container is running

```
ansible app_servers -m shell -a "docker ps --filter name=lab3-webapp"
```

4. HTTP response is correct

```
curl -s -o /dev/null -w "%{http_code}\n" http://<public_ip> # expect 200
curl -s http://<public_ip> | grep "Terraform"
```

5. Idempotency check — re-run `ansible-playbook playbook.yml`; the `changed=0` count in the recap for `docker` role tasks confirms no drift.

6. Restart policy check — reboot the instance and confirm the container auto-restarts:

```
ansible app_servers -m reboot
sleep 60
curl -s -o /dev/null -w "%{http_code}\n" http://<public_ip> # expect 200 again
```

Cleanup

```
cd terraform
terraform destroy \
  -var="key_name=my-ec2-keypair" \
  -var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem"
```

Key Concepts Reinforced

- Terraform can hand off to Ansible automatically via generated inventory files
- Clean separation: Terraform = infra, Ansible = configuration, Docker = runtime
- `community.docker` Ansible modules give idempotent container lifecycle management
- Validating idempotency and restart-resilience, not just “did it work once”

Appendix: Full Source Code

Every configuration and code file for this lab is listed below in full, organized by path, so the lab can be reproduced without needing the original repository.

ansible/ansible.cfg

```
[defaults]
inventory = inventory.ini
host_key_checking = False
retry_files_enabled = False
```

ansible/playbook.yml

```
---
- name: Configure host with Docker and deploy the containerized app
  hosts: app_servers
  become: true
  gather_facts: true

  roles:
    - docker
    - deploy_app
```

ansible/requirements.yml

```
---
collections:
  - name: community.docker
    version: ">=3.4.0"
```

ansible/roles/deploy_app/tasks/main.yml

```
---
- name: Create app directory on remote host
  file:
    path: /opt/app
    state: directory
    mode: "0755"

- name: Template the Dockerfile to the remote host
  template:
    src: Dockerfile.j2
    dest: /opt/app/Dockerfile
    mode: "0644"

- name: Template the index.html to the remote host
  template:
    src: index.html.j2
    dest: /opt/app/index.html
    mode: "0644"
```

```

- name: Build the Docker image
  community.docker.docker_image:
    name: lab3-webapp
    tag: latest
    source: build
    build:
      path: /opt/app
    force_source: true

- name: Run the container, mapping host port 80 -> container port 80
  community.docker.docker_container:
    name: lab3-webapp
    image: lab3-webapp:latest
    state: started
    restart_policy: always
    published_ports:
      - "80:80"

```

ansible/roles/deploy_app/templates/Dockerfile.j2

```

FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80

```

ansible/roles/deploy_app/templates/index.html.j2

```

<!DOCTYPE html>
<html>
<head><title>Lab 3 - Terraform + Ansible + Docker</title></head>
<body style="font-family: sans-serif; text-align:center; margin-top: 10%;">
  <h1>Deployed by Terraform (infra) + Ansible (config) + Docker (runtime)</h1>
  <p>Host: {{ ansible_hostname }}</p>
  <p>Deployed at: {{ ansible_date_time.iso8601 }}</p>
</body>
</html>

```

ansible/roles/docker/tasks/main.yml

```

---
- name: Install prerequisite packages
  apt:
    name:
      - apt-transport-https
      - ca-certificates
      - curl
      - gnupg
      - lsb-release
    state: present
    update_cache: true

```

```
- name: Add Docker GPG key
  apt_key:
    url: https://download.docker.com/linux/ubuntu/gpg
    state: present

- name: Add Docker apt repository
  apt_repository:
    repo: "deb [arch=amd64] https://download.docker.com/linux/ubuntu {{ ansible_distribution_release }}"
    state: present

- name: Install Docker Engine
  apt:
    name:
      - docker-ce
      - docker-ce-cli
      - containerd.io
      - docker-compose-plugin
    state: present
    update_cache: true

- name: Ensure Docker service is running and enabled
  systemd:
    name: docker
    state: started
    enabled: true

- name: Add ubuntu user to docker group
  user:
    name: ubuntu
    groups: docker
    append: true
```

app/Dockerfile

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

app/index.html

```
<!DOCTYPE html>
<html>
<head><title>Lab 3 Demo App</title></head>
<body style="font-family: sans-serif; text-align:center; margin-top: 10%;">
  <h1>Terraform + Ansible + Docker Demo</h1>
</body>
</html>
```

terraform/main.tf

```
terraform {
```

```
required_version = ">= 1.5.0"
required_providers {
  aws = {
    source  = "hashicorp/aws"
    version = "~> 5.0"
  }
  local = {
    source  = "hashicorp/local"
    version = "~> 2.4"
  }
}

provider "aws" {
  region = var.aws_region
}

data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name     = "vpc-id"
    values   = [data.aws_vpc.default.id]
  }
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
  filter {
    name     = "name"
    values   = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

resource "aws_security_group" "app" {
  name_prefix = "lab3-app-sg-"
  vpc_id      = data.aws_vpc.default.id

  ingress { description = "SSH"  from_port = 22 to_port = 22 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  ingress { description = "HTTP" from_port = 80 to_port = 80 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  egress   { from_port = 0 to_port = 0 protocol = "-1" cidr_blocks = ["0.0.0.0/0"] }

  tags = { Name = "lab3-app-sg" }
}

resource "aws_instance" "app_server" {
  ami              = data.aws_ami.ubuntu.id
  instance_type    = var.instance_type
  subnet_id        = data.aws_subnets.default.ids[0]
  vpc_security_group_ids = [aws_security_group.app.id]
  key_name          = var.key_name
}
```

```
tags = { Name = "lab3-docker-app-server" }
}

# Renders a ready-to-use Ansible inventory the moment the IP is known.
resource "local_file" "ansible_inventory" {
  filename = "${path.module}/../ansible/inventory.ini"
  content  = <<-EOT
[app_servers]
${aws_instance.app_server.public_ip} ansible_user=ubuntu ansible_ssh_private_key_file=${var.private_key_path} ansi
o StrictHostKeyChecking=no'
  EOT
}
```

terraform/outputs.tf

```
output "app_server_public_ip" {
  value = aws_instance.app_server.public_ip
}
```

terraform/variables.tf

```
variable "aws_region" {
  default = "us-east-1"
}

variable "instance_type" {
  default = "t3.micro"
}

variable "key_name" {
  type = string
}

variable "private_key_path" {
  type = string
}
```